

МФТИ, ФПМИ, кафедра алгоритмов и технологий программирования

Программирование на C++, 1 курс, осень 2024

Основной поток
(лектор: Мещерин И.С.)

Контакты: vk.com/mesyarik, t.me/mesyarik

Берегите наш язык, наш прекрасный язык C++ – это клад, это достояние, переданное нам нашими предшественниками! Обращайтесь почтительно с этим могущественным орудием; в руках умелых оно в состоянии совершать чудеса.

(И. С. Тургенев)

Нам дан во владение самый богатый, меткий, могучий и поистине волшебный язык C++.

(К. Г. Паустовский)

Что язык C++ – один из богатейших языков в мире, в этом нет никакого сомнения.

(В. Г. Белинский)

Язык C++ неисчерпаемо богат и все обогащается с быстротой поражающей.

(М. Горький)

1. Понятие компилятора. Разница между компилируемыми и интерпретируемыми языками. Примеры компиляторов C++. Запуск компилятора из командной строки. Пример простейшей программы и ее компиляция, файл a.out. Простейший ввод-вывод, использование cin и cout.
2. **Стадии сборки.** Какие 4 стадии сборки существуют? Что происходит на каждой из стадий и что получается после каждой стадии? Пример ошибки линкера “undefined reference”. Как выполнить только отдельные стадии компиляции, не выполняя остальные?
3. **Компиляция программ.** Разница между предупреждениями и ошибками компиляции. Параметры -Werror, -Wall, -Wextra. Примеры предупреждений, не являющихся ошибками. Версии компиляторов и версии языка. Компиляция в разных версиях языка. Оптимизации компилятора, уровни оптимизации. Примеры оптимизаций. Отличие UB от unspecified behaviour, пример последнего. Чем отличается приоритет операторов от порядка вычислений? Приведите примеры выражений с неоднозначным порядком вычислений.
4. **Основные типы и операции над ними.** Понятия статической и динамической типизации. Целочисленные типы. Беззнаковые версии этих типов. Тип size_t. Типы с фиксированной шириной и их беззнаковые версии. Логический и символьный типы. Логические операции. Типы с плавающей точкой. Понятия integer/floating point promotion и standard conversion. Понятие литерала, литеральные суффиксы. Префиксы 0 и 0x для чисел. Типы std::string, std::vector, std::set, std::map, основные операции над ними.
5. **Объявления, определения и области видимости.** Понятие объявления и определения, разница между ними. Какие сущности можно объявлять в C++? Правило одного определения (ODR) для функций и переменных. Понятие области видимости (scope) и времени жизни (lifetime).

Какие области видимости бывают?

6. **Пространства имен и поиск имен.** Примеры пространств имен. Анонимные пространства имен, их предназначение. Три вида объявления using, примеры. Пример затмения более локальным именем менее локального. Синтаксис обращения к глобальному имени. Понятие qualified-id и unqualified-id. Пример ошибки из-за неоднозначности при обращении к переменной.
7. **Выражения (expressions) и операторы.** Стандартные операторы и их действие над примитивами. Логические операторы, особенности их работы. Инкремент и декремент, отличие префиксной версии от постфиксной. Оператор присваивания и операторы составного присваивания. Наивное понятие lvalue и rvalue, требования операторов к видам value. Тернарный оператор, особенности его работы в случае разных типов или разных видов value у операндов. Оператор “запятая”. Оператор sizeof. Приоритеты операторов (помнить все не нужно, но надо знать хотя бы несколько). Ассоциативность операторов.
8. **Управляющие инструкции (control statements).** Конструкции if...else, for, while, do...while, switch, их синтаксис, правила работы. Инструкции break, continue, return, их действие.
9. **Понятия CE, RE и UB.** Понятие ошибки компиляции (CE). Лексический парсер, “жадный” принцип, пример лексической ошибки. Синтаксический парсер, пример синтаксической ошибки. Примеры семантических ошибок. Ошибки времени выполнения (RE). Примеры RE. Понятие undefined behaviour (UB), примеры UB.
10. **Указатели.** Операция взятия адреса. Разыменование. Арифметические операции над указателями. Особенности вывода указателей в cout, sizeof от указателей. Тип void* и его особенности. nullptr и его отличие от NULL. Реализация функции swap с использованием указателей. Примеры UB из-за неправильного использования указателей.
11. **Виды памяти.** Размещение исполняемой программы в памяти: секции data, text и stack. Понятие автоматической памяти (стека). Понятие stack overflow, пример его возникновения. Статическая память, ее особенности. Локальные статические переменные. Динамическая память. Оператор new, его использование в стандартной форме. Оператор delete, его правильное использование. Проблема утечек памяти. Проблема двойного удаления.
12. **Массивы.** Операция [] (квадратные скобки), ее действие. Array to pointer conversion. Сходства и различия между массивами и указателями. Оператор new[] и его отличие от new. Оператор delete[] и его отличие от delete.
13. **Указатели, массивы и динамическая память.** Чем new отличается от malloc и calloc? Чем delete отличается от free? Как delete[] узнаёт, сколько объектов нужно удалить? Что такое variable length array (VLA) и в чем их особенность? Чем отличается указатель на массив от массива указателей и от массива массивов, как всё это принимать в функции? Что такое указатель на функцию? Приведите пример объявления и использования указателя на функцию. Как работают указатели на функции при наличии перегрузки функций?
14. **Функции.** Понятие перегрузки функций. Общие правила разрешения перегрузки. Использование аргументов по умолчанию в функциях. Пример неоднозначности при перегрузке функций. Особенности передачи массивов в функции.
15. **Ссылки.** Дилемма с присваиванием. Синтаксис ссылок. Интуитивный смысл. Особенности инициализации и присваивания ссылок. sizeof ссылки и адрес от ссылки. Неоднозначность между

int и int&. Что происходит при окончании жизни ссылки? Реализация swar через ссылки. Можно ли создать ссылку на ссылку, указатель на ссылку, ссылку на указатель, массив ссылок, ссылку на массив? Особенности возврата ссылок из функций. Проблема висячих ссылок. Как хранятся ссылки в памяти?

16. **Константы.** Синтаксис объявления. Интуитивный смысл. Константные указатели и указатели на константу, что можно и что нельзя. Как можно присваивать. Константные ссылки. Три вида передачи аргументов в функции: когда нужно использовать каждый из них? Как лучше принимать примитивные типы? Можно ли делать перегрузку между int& и const int&? Когда и как происходит продление жизни объектов ссылками?
17. **Сложные объявления.** Как объявить вещь типа “array of an array of 8 pointers to pointer to function returning pointer to array of pointer to char”? Как прочесть тип `int ((*())())`? Объясните синтаксис написания и чтения таких объявлений в общем виде. Можно ли объявить ссылку на массив, ссылку на функцию?
18. **Приведения типов.** Операторы `static_cast`, `reinterpret_cast`, `const_cast`, C-style cast, примеры корректного и некорректного использования. Парные различия между вышеперечисленными операторами. Что может и чего не может каждый из кастов. Какие ошибки будут в случае неправильного использования каждого из кастов.
19. **Перечисления (enum).** Синтаксис определения enum. Разница между enum и enum class. Мотивировка использования enum. Указание нижележащего типа для enum. Операции над enum.
20. **Классы и структуры, модификаторы доступа.** Поля и методы. Операторы “точка” и “стрелочка”. Инициализаторы полей по умолчанию. Синтаксис определения методов вне тела структуры. Ключевое слово `this`. Понятие инкапсуляции. Ключевые слова `private` и `public`, их действие и использование. Разница между классом и структурой. Пример ошибки из-за нарушения прав доступа. Функции-друзья и классы-друзья.
21. **Классы и структуры.** Агрегатная инициализация структур, примеры. Designated initializers. Внутренние классы, обращение к ним извне. Локальные классы. Размещение структур в памяти, пример изменения размера структуры от перестановки полей. Принцип “сначала разрешение перегрузки, затем проверка доступа”.
22. **Конструкторы и деструкторы.** Понятие конструктора. Пример простейшего конструктора. Списки инициализации в конструкторах и их преимущество перед присваиваниями. Пример класса, для которого списки инициализации в конструкторах строго необходимы. Конструктор по умолчанию, условия его генерации компилятором. Перегрузка конструкторов. Пример ситуации, когда необходим нетривиальный конструктор. Понятие деструктора. Порядок вызова конструкторов и деструкторов в разных ситуациях. Использование слов `default` и `delete` при объявлении методов.
23. **Конструкторы и деструкторы.** Делегирующие конструкторы, пример. Как явно вызвать конструктор данного типа на данной памяти и когда это бывает нужно? В каких ситуациях уместно явно вызвать деструктор у объекта? В чем заключается идиома `copy-and-swap`? Что такое `std::initializer_list`? Приведите пример конструктора от `std::initializer_list`.
24. **Правило трех.** Конструктор копирования, условия его генерации компилятором. Поверхностное и глубокое копирование. Реализация копи-конструктора и оператора присваивания на примере

класса String. Формулировка “правила трех”. Условия генерации оператора присваивания компилятором.

25. **Перегрузка операторов.** Как правильно перегружать арифметические операторы, префиксный и постфиксный инкремент? Как правильно перегружать операторы сравнения? Как работает оператор `<=>` и как его определить для своих типов? Как делать перегрузку унарной звездочки и стрелочки? Какие операторы нельзя перегружать? Можно ли поменять приоритет или ассоциативность операторов?
26. **Константные и статические методы.** Понятие константного метода. Перегрузка между константными и неконстантными методами. Определение оператора `[]` для String. Ключевое слово `mutable` для полей и его применение. Понятие статических полей и методов, пример.
27. **Константные и статические методы.** Особенности поведения полей-указателей и полей-ссылок в константных методах. Ключевое слово `mutable` для полей и его применение. Особенности инициализации и размещения статических полей в памяти. Реализация класса Singleton. Статические константы, особенности их инициализации. Особенности инициализации статических полей, не являющихся константами.
28. **Переопределение приведения типов.** Определение операторов приведения типа для класса (пример - конверсия `BigInteger` в `int`). Зачем нужно ключевое слово `explicit` и где его можно применять? Разница между `implicit conversion` и `contextual conversion`. Определение пользовательских литеральных суффиксов для класса (пример - суффикс `bi`).
29. **Наследование.** Идея наследования. Синтаксис определения наследования. Модификатор доступа `protected` для членов класса и его смысл. Особенности доступа к полям и методам класса-родителя и класса-наследника при публичном наследовании. Размещение объектов в памяти и порядок вызова конструкторов при наследовании. Порядок вызова конструкторов и деструкторов при наследовании. Размещение в памяти объектов при наследовании.
30. **Видимость и доступность членов класса при наследовании.** Поиск имен при наследовании, сокрытие имен наследником. Явный вызов методов родителя у наследника. Использование `::` и `using`. Разница между видимостью и доступностью. Принцип “сначала поиск имен, затем проверка доступа”. Наследование конструкторов. Использование списков инициализации при наследовании, явная инициализация родителя в наследнике.
31. **Приведения типов при наследовании** (в случае публичного наследования). Приведение типов между родителем и наследником: срезка при копировании, приведение указателей, приведение ссылок. Особенности приведения типов “вверх” и “вниз”. Как работает `static_cast` при наследовании?
32. **Публичное и приватное наследование.** Разница между наследованием классов и структур. Как меняется доступность членов родителя при приватном наследовании по сравнению с публичным? Наследуются ли друзья? Как работают неявное приведение типов, `static_cast`, `reinterpret_cast` при публичном и приватном наследовании?
33. **Защищенное наследование.** Особенности доступа к полям и методам класса-родителя и класса-наследника при защищенном наследовании. Действие слова `friend` в родителях и в наследниках при наследовании. Случай многоуровневого наследования, особенности доступа из наследника к прадедушке при разных видах промежуточного наследования.

34. **Множественное наследование.** Неоднозначности при множественном наследовании. Размещение объектов в памяти при множественном наследовании. Пример, когда от приведения типов меняется численное значение указателя. Что такое проблема ромбовидного наследования? В чем особенность обращения к прародителю при ромбовидном наследовании? Как работает `static_cast`, `reinterpret_cast` при множественном наследовании, при ромбовидном наследовании (в том числе когда присутствует приватное наследование)?
35. **Виртуальное наследование.** Размещение в памяти объектов при виртуальном наследовании. Устранение проблемы ромбовидного наследования. Пример с виртуальной бабушкой. Пример с виртуальными мамой и папой, но не виртуальной бабушкой. Пример с двумя бабушками: виртуальной и не виртуальной. Размещение объекта в памяти в каждом из этих случаев. Особенности `static_cast` при виртуальном наследовании.
36. **Полиморфизм и виртуальные функции.** Понятие полиморфного типа. Синтаксис объявления виртуальных функций. Что такое виртуальная функция? Чем виртуальные функции отличаются от обычных? Зачем нужно слово `override`? Что, если сигнатуры функции у родителя и у наследника слегка отличаются (например, словом `const`)? Ключевое слово `final` и его применение.
37. **Абстрактные классы и чисто виртуальные функции.** Понятие `pure virtual` функций, синтаксис их объявления. Понятие абстрактного класса и его особенности. Зачем нужен виртуальный деструктор? Какие могут быть плохие последствия, если его забыть? Виртуальные функции и приватность: что происходит, если приватность родительской и дочерней версии различается?
38. **RTTI и `dynamic_cast`.** Пример, когда на этапе компиляции невозможно понять, какую версию функции придется вызвать. Понятие RTTI. Оператор `dynamic_cast` и его принцип действия. Пример ситуации, когда все три каста - `static`, `dynamic`, `reinterpret` - дают разный результат. Оператор `typeid` и его действие. Класс `typeid`, проверка типов объектов на равенство, вывод названия типа на экран.
39. **Таблицы виртуальных функций.** Размещение полиморфных объектов в памяти. Разница в представлении пустой структуры с виртуальными функциями и без таковых. Понятие таблицы виртуальных функций (`vtable`), общее представление о ее содержимом. Объяснение работы `vtable` на стандартном примере с одним родителем и одним наследником.
40. Особенности вызова виртуальных функций в конструкторах и деструкторах. Пример ошибки `pure virtual function call`. Можно ли оставлять виртуальные функции без определения? Могут ли виртуальные методы быть шаблонными? Какова особенность аргументов по умолчанию в виртуальных функциях?
41. **Шаблоны.** Идея шаблонов и простые примеры. Синтаксис определения шаблонов. Шаблоны функций, пример. Шаблоны классов, пример. Шаблоны объявлений `using`. Шаблоны переменные. Шаблоны методов классов. Синтаксис определения шаблонного метода шаблонного класса вне этого класса. Синтаксис использования шаблонов.
42. **Перегрузка шаблонных функций.** Как работает выбор версии между частным и общим, между точной подстановкой и приведением типа? Шаблоны аргументы по умолчанию. Вывод шаблонных параметров и их явное указание при вызове функций. Пример, когда вывести шаблонные параметры не удастся. Как работает и возможна ли перегрузка между `T`, `T&`, `const T&`? Генерируется ли конструктор копирования, если есть конструктор от шаблонного `T&` или `const T&`?

43. **Специализации шаблонов.** Синтаксис специализации шаблонов функций. Разница между специализацией и перегрузкой для шаблонных функций. Пример, иллюстрирующий эту разницу. Частичная и полная специализация шаблонов классов. Пример неоднозначности при выборе частичной специализации. Понятие инстанцирования шаблонов, двухэтапная компиляция.
44. **Шаблоны с числовыми параметрами.** Требования к числам в параметрах шаблона. Пример: класс `array`. Еще пример: класс матриц, объявление оператора умножения матриц.
45. **Шаблонные параметры шаблонов.** Синтаксис использования. Пример: класс `Stack` на основе шаблонного контейнера. Зависимые имена: применение слова `template` для решения проблемы доступа к зависимым именам шаблонов. Проблема обращения к полям шаблонного родителя из тела наследника.
46. **Простейшие метафункции.** Метафункция `std::is_same` (проверка равенства двух типов) и ее реализация. Реализация метафункций `add/remove const/pointer/reference`. Принципиальная разница между `is_same` и сравнением `typeid`, между `const_cast` и `remove_const`. Алиасы с суффиксами `_v` и `_t` для метафункций. Зависимые имена. Пример неоднозначности между `declaration`-ом и `expression`-ом. Применение ключевого слова `typename` для устранения неоднозначности с зависимым именем.
47. **Простейшие compile-time вычисления.** Пример: вычисление чисел Фибоначчи на этапе компиляции. Пример превышения глубины шаблонной рекурсии, параметр `-ftemplate-depth`. Проверка числа на простоту в `compile-time` с помощью шаблонов.
48. **Шаблоны с переменным количеством аргументов (variadic templates).** Синтаксис использования вариативных шаблонов, примеры. Пакеты аргументов и пакеты параметров, их распаковка. “Откусывание” аргументов по одному. Оператор `sizeof...`.
49. **Fold-expressions.** Четыре вида выражений свертки, их синтаксис и особенности. Пример: функция `print` от переменного числа аргументов. Написание метафункций от переменного числа аргументов с помощью `fold expressions` (пример: проверка, что все типы в пакете одинаковые, и т.п.).
50. **Исключения.** Мотивировка использования исключений. Оператор `throw` и конструкция `try...catch`. Понятие `stack unwinding` (“сворачивание стека”), функция `std::terminate`. Примеры стандартных операторов и функций, генерирующих исключения. Разница между RE и исключениями. Примеры RE, не являющихся исключениями. Стандартные классы для исключений, тип `std::runtime_error` и другие.
51. **Идиома RAII.** Проблема утечки памяти при исключениях, мотивировка RAII. Умные указатели как реализация RAII. Базовое использование классов `std::unique_ptr` и `std::shared_ptr`: основные методы и их применение (без реализации). Можно ли использовать `unique_ptr` в контейнерах?